Decoupling Large Folios from CONFIG_THP

Pankaj Raghav
Global Open Source Team
Samsung

CONFIG_THP in blkdev.h

- Found a **NULL ptr** dereference when a block device logical **block size > PAGE_SIZE** (LBS configurations) with **THP disabled.**

```
diff --git a/include/linux/blkdev.h b/include/linux/blkdev.h
index 82348fcc2455..724ddc40f35c 100644
--- a/include/linux/blkdev.h
+++ b/include/linux/blkdev.h
@@ -270,11 +270,16 @@ static inline dev_t disk_devt(struct gendisk *disk)
     return MKDEV(disk->major, disk->first_minor);
 }

+#ifdef CONFIG_TRANSPARENT_HUGEPAGE
 /*
  * We should strive for 1 << (PAGE_SHIFT + MAX_PAGECACHE_ORDER)
  * however we constrain this to what we can validate and test.
  */
 #define BLK_MAX_BLOCK_SIZE      SZ_64K
+#else
+#define BLK_MAX_BLOCK_SIZE      PAGE_SIZE
+#endif
+
```

CONFIG_THP in blkdev.h

- Found a **NULL ptr** dereference when a block device logical **block size > PAGE_SIZE** (LBS configurations) with **THP disabled.**

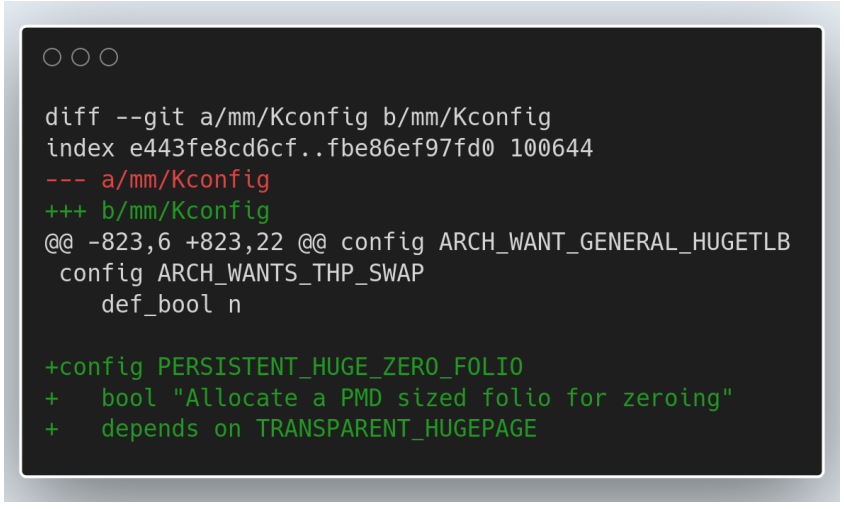
I mean, it's a **quick fix**, so sure. But really we **should remove that dependency.** Particularly for block size > Page Size as there's no need for any of the splitting code.

- Willy

```
diff --git a/include/linux/blkdev.h b/include/linux/blkdev.h
index 82348fcc2455..724ddc40f35c 100644
--- a/include/linux/blkdev.h
+++ b/include/linux/blkdev.h
@@ -270,11 +270,16 @@ static inline dev_t disk_devt(struct gendisk *disk)
     return MKDEV(disk->major, disk->first_minor);
 }

+#ifdef CONFIG_TRANSPARENT_HUGEPAGE
 /*
  * We should strive for 1 << (PAGE_SHIFT + MAX_PAGECACHE_ORDER)
  * however we constrain this to what we can validate and test.
  */
 #define BLK_MAX_BLOCK_SIZE      SZ_64K
+#else
+#define BLK_MAX_BLOCK_SIZE      PAGE_SIZE
+#endif
+
```

More CONFIG_THP confusion



```
diff --git a/mm/Kconfig b/mm/Kconfig
index e443fe8cd6cf..fbe86ef97fd0 100644
--- a/mm/Kconfig
+++ b/mm/Kconfig
@@ -823,6 +823,22 @@ config ARCH_WANT_GENERAL_HUGETLB
 config ARCH_WANTS_THP_SWAP
    def_bool n

+config PERSISTENT_HUGE_ZERO_FOLIO
+   bool "Allocate a PMD sized folio for zeroing"
+   depends on TRANSPARENT_HUGEPAGE
```

I feel like we really need to **sort out** what is/isn't **predicated on THP**... it seems like **THP is sort of short hand for 'any large folio stuff' but not always…**

\- Lorenzo

Large folio and THP dependency

- Here **large folios** refer to the one created by the **page cache** (file-backed)
  - **Not** anonymous large folios (**mTHP**)

- Large folios depended on **some** THP infrastructure when they were introduced.

- Currently, the main feature it depends on is the **splitting** infrastructure in THP.

- For example, splitting is done during truncate or under memory pressure, etc.

```
[PATCH] mm: Make large folios depend on THP

Some parts of the VM still depend on THP to handle large folios
correctly.  Until those are fixed, prevent creating large folios
if THP are disabled.

Signed-off-by: Matthew Wilcox (Oracle) <willy@infradead.org>
---
 include/linux/pagemap.h | 7 ++++++-
 1 file changed, 6 insertions(+), 1 deletion(-)

diff --git a/include/linux/pagemap.h b/include/linux/pagemap.h
index 0a2417fc531c9..20d7cbabf6546 100644
--- a/include/linux/pagemap.h
+++ b/include/linux/pagemap.h
@@ -306,9 +306,14 @@ static inline void mapping_set_large_folios(struct address_space *mapping)
     __set_bit(AS_LARGE_FOLIO_SUPPORT, &mapping->flags);
 }

+/*
+ * Large folio support currently depends on THP.  These dependencies are
+ * being worked on but are not yet fixed.
+ */
 static inline bool mapping_large_folio_support(struct address_space *mapping)
 {
-    return test_bit(AS_LARGE_FOLIO_SUPPORT, &mapping->flags);
+    return IS_ENABLED(CONFIG_TRANSPARENT_HUGEPAGE) &&
+        test_bit(AS_LARGE_FOLIO_SUPPORT, &mapping->flags);
 }

 static inline int filemap_nr_thps(struct address_space *mapping)
--
2.51.0
```

Large folio and THP dependency

Issues:

- CONFIG_THP are spilling over to lower level subsystems such as block layer and Filesystems.

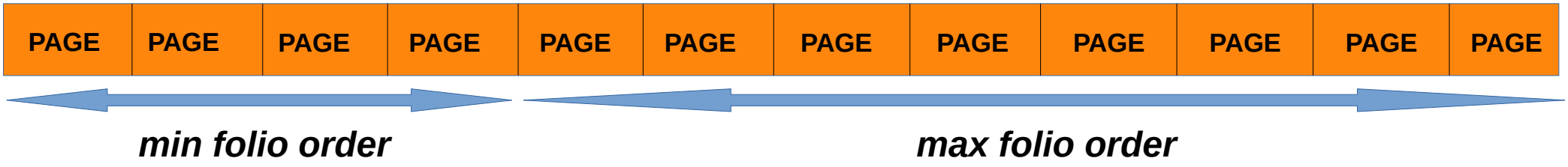
- This dependency is very unclear.

During the review of Large block size support for ext4:

> It **really** isn't obvious that **large block size** support and **transparent hugepages** are **linked**.
>
> - Ted

Solution 1

No splitting when THP is disabled



- Folio order can be controlled by **min** and **max folio order.**

- Folio cannot be split lower than min order

No splitting when THP is disabled



- Set **min folio order == max folio order**

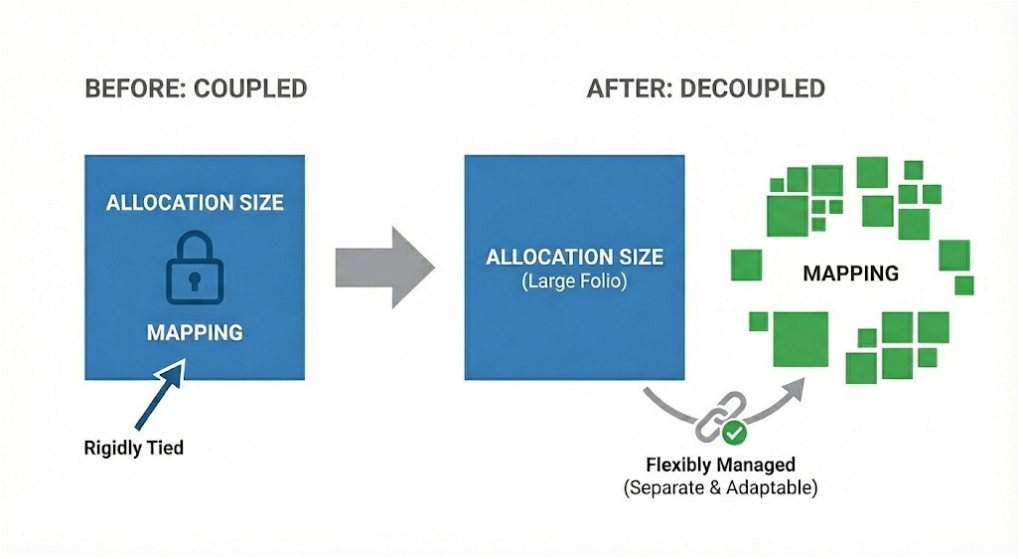
- Do **not** use the **splitting infra** for large folios when **THP** is **disabled**

- Not complicated. Requires minimal changes.

- I sent a RFC of this approach here: link

Solution 2

Separating allocation from mapping



- Complete overhaul by decoupling the **size to allocate** from **how can it be mapped**
- Move the **splitting code** to a separate **file** that is not **dependent on THP**
- When **THP** is **disabled**, allow **PMD allocation** but do not **map** them through a **PMD**

Questions?

Acknowledgements

- Thanks to David Hildenbrand, Zi Yan and Matthew Willcox for all their help.